

Compte-rendu TP6

C - Composants périphériques

C1

Le composant PibusMultiTimer est configurable en logiciel, il est donc une cible sur le bus pour permettre de le configurer en accédant à son registre de mask.

L'argument 'ntimer' de ce composant permet de définir le nombre de timers programmables du composant. Cela va également définir le nombre de signaux d'interruptions en sortie.

Les registres adressables de ce composant sont:

- *r_value*:
Adresse: $\text{SEG_TIM_BASE} + n_timer * 0x10$.
Permet de définir ou de lire la valeur du timer.
- *r_running*:
Adresse: $\text{SEG_TIM_BASE} + 0x4 + n_timer * 0x10$.
Permet d'activer (1) ou de désactiver (0) l'incrémentation du timer.
- *r_period*: Adresse:
 $\text{SEG_TIM_BASE} + 0x8 + n_timer * 0x10$.
Permet de définir la période d'incrémentacion du timer en npmbre de cycles.
- *r_irq*:
Adresse: $\text{SEG_TIM_BASE} + 0xC + n_timer * 0x10$.
Permet d'activer (1) ou désactiver (0) les interruptions.

C2

Le composant PibusIcus est une cible sur le bus car il permet de programmer les interruptions. Ce sont donc les processeurs qui vont faire des transactions sur le bus pour le programmer.

L'argument *nirq* du constructeur permet de définir le nombre d'interruptions que l'on veut gérer en entrée de l'ICU.

L'argument *nproc* définit le nombre de processeur connectés à l'ICU, et donc le nombre de sorties de l'ICU.

Un logiciel peut aiguiller les interruptions de l'ICU à l'aide d'un mask pour chacun des processeurs. Ces masks permettent aux processeurs de choisir les interruptions qui les intéressent.

Les registres adressables de ce composant sont:

- *r_int*:
Adresse: $\text{SEG_ICU_BASE} + 0x0 + n_proc * 0x20$
Permet de lire le status des lignes d'interruptions.

- *r_mask*:
Adresse: $\text{SEG_ICU_BASE} + 0x4 + n_proc * 0x20$
Permet de lire le mask des IRQ associé au processeur.
- *icu_set*:
Adresse: $\text{SEG_ICU_BASE} + 0x8 + n_proc * 0x20$
Permet d'écrire les bits du mask IRQ.
- *icu_clear*:
Adresse: $\text{SEG_ICU_BASE} + 0xC + n_proc * 0x20$
Permet de mettre à 0 les bits du mask IRQ.
- *icu_highest*:
Adresse: $\text{SEG_ICU_BASE} + 0x10 + n_proc * 0x20$
Permet de lire le numéro de la ligne d'IRQ de priorité maximale.

C3

L'adresse associée au composant PibusIcu doit être alignée sur $32 * 8$ octets. Ceci est dû au fait que les MSB sont codés sur 3 bits donc $2^5 = 32$ possibilités avec les LSB qui sont codés sur $2^3 = 8$. On a donc une adresse de base qui est alignée sur 8, il suffit donc simplement de comparer les MSB (bits de poids fort) pour vérifier de quel composant il s'agit. Si on relâchait cette contrainte, on devrait alors comparer également les bits de poids faible (LSB), donc 3 bits, afin de déterminer à quel composant appartient une adresse.

C4

Pour relier les 4 lignes d'interruptions, les ports IRQ_IN du contrôleur ICU sont connectés à des signaux internes qui sont eux-mêmes connectés aux composants PibusMultiTimer et PibusMultiTty. Par exemple, les signaux internes reliant le timer à l'icu sont les deux premières entrées du tableau *signal_irq_tim* et pour ceux du tty ils sont dans *signal_irq_tty_put* et *signal_irq_tty_get*.

D - Lancement des tâches

D2

L'adresse de *main_prime()* est 0x004012e8.

L'adresse de *main_pgcd()* est 0x004013fc.

D3

On force GCC à construire la table de saut au début du segment data en le spécifiant dans le fichier *app.ld* qui va spécifier au linker la construction de cette table. C'est la ligne `"*(.ctors)"` qui permet de dire au linker que l'on veut mettre tous les constructeurs au début du segment *seg_data*.

On déclare également *main* comme constructeur avec la syntaxe ***attribute((constructor))*** *main*. (vu dans *app.ld*)

D4

Le programme de calcul du PGCD attend une réponse de l'utilisateur, il attend donc une interruption. Pour cela il faut donc configurer l'ICU afin de gérer les interruptions multi processeurs.

E - Activation du Timer

E1

Pour se brancher à une routine ISR, le processeur initialise un tableau contenant les pointeurs des ISR de chaque IRQ, c'est `_interrupt_vector`.

Entre le branchement au point d'entrée d'adresse 0x80000180 et le branchement à la routine Timer:

`_giet` récupère dans `_cause_vector`, l'élément indexé par le XCODE trouvé dans `register_cause`. Cela lui permet d'analyser si il doit gérer une erreur, une interruption ou autre chose et de sauter à la fonction correspondante. Dans notre cas on va à `_int_handler` qui gère les interruptions et va appeler la fonction `_int_demux`. Cela permet d'accéder au vecteur des ISR et de sauter à l'ISR correspondant à l'interruption.

E2

La routine d'interruption `_isr_timer`, qui se trouve dans le fichier `irq_handler.c`, reset l'IRQ et affiche l'heure et la date de la réception de l'interruption.

E3

On accède au tableau `_interrupt_vector` en faisant `lw $27, _interrupt_vector`.

Puis on lit l'adresse de l'ISR `lw $29, _isr_timer`.

Enfin on détermine le bon décalage dans le tableau avec le calcul suivant:

$\$Timer_i \Rightarrow 2 + 2 \times i\$$

$\$i = 0 \Rightarrow 2 + 2 \times 0 = 2\$$

$\$i = 1 \Rightarrow 2 + 2 \times 1 = 4\$$

Puis on multiplie par 4 car c'est un vecteur d'adresses codées sur 4 octets.

Cela nous donne donc un décalage de $\$2 \times 4 = 8\$$ pour le processeur 0 et $\$4 \times 4 = 16\$$ pour le processeur 1.

E4

On récupère l'adresse de base du composant `multi_timer`, puis on incrémente de 16 pour le processeur 1 obtenir celle du timer 1 car chaque timer est sur 16 octets.

E5

Pour obtenir l'adresse de ICU0 et ICU1:

ICU0 correspond à l'adresse de base de ICU.

ICU1 correspond à l'adresse de base de ICU + 32 car chaque ICU est sur 32 octets.

On incrémente ensuite chaque adresse de 8 pour obtenir l'adresse du registre correspondant au `ICU_SET`.

E6

- Le processeur 0 écrit la première valeur dans le vecteur d'interruption au cycle 43.
Le premier accès mémoire correspond à la première occurrence de "`sel_ram = 1`".

la \$27, _interrupt_vector

- Le registre *MASK[0]* est configuré au cycle 56.
On identifie la première occurrence de "*sel_icu* = 1".

```
sw $29, 8($27)
```

- Le registre *TIMER[0]* est configuré au cycle 60.
On identifie la première occurrence de "*sel_tim* = 1".

```
sw $29, 8($27) # period = 50000
```

E7

- Le processeur 0 reçoit la première interruption du *TIMER[0]* au cycle 50067.
On identifie la première occurrence de "*tim_irq[0]* = 1".

F - Activation des interruptions TTY

F1

Lors de l'exécution d'un programme utilisateur, si celui-ci lit le buffer temporaire, pour assurer la cohérence mémoire on ne doit pas avoir de valeur asynchrone. S'il le programme lit deux fois le buffer et que la valeur change, cela pourrait poser problème.

On règle donc cette incohérence en utilisant la fonction *_tty_read_irq* pour mettre à jour le buffer.

F2

L'utilisateur écrit un caractère, une interruption est déclenchée provoque l'appel de l'ISR *ist_tty_get* par le composant TTY qui va mettre le caractère dans le *_tty_get_buf* et mettre *_tty_get_full* à 1.

Avec giet on aura donc Ensuite, le logiciel vérifiera et lira la valeur grâce à l'appel système *tty_read_irq* afin de l'enregistrer dans un buffer utilisateur.

Les fonctions appelées seront donc les suivantes:

_giet -> *_int_handler* -> *_int_demux* -> *_isr_tty_get* -> *_isr_tty_get_indexed* Puis *_tty_read_irq* sera appelée par le logiciel de manière synchrone pour récupérer le caractère.

F3

D'après le code décrit dans *irq_handler.c*, si le tampon *_tty_get_buf* est plein au moment de l'exécution de l'ISR, celui-ci est réécrit par la nouvelle valeur lue.

F4

Fonction *_tty_read_irq()*:

Lit le buffer *_tty_get_full* du composant TTY pour vérifier si un nouveau caractère est disponible.

Si aucun caractère n'est disponible (*_tty_get_full* == 0), retourne 0 et ne fait rien.

Si un caractère est disponible (*_tty_get_full* == 1), enregistre le contenu du buffer *_tty_get_buf* dans un buffer passé en argument. Pour signaler qu'un caractère a été lu, met à 0 *_tty_get_full* et retourne 1.

Le numéro du TTY est calculé en récupérant le procid qui permet de récupérer le *task_id* puis le *tty_id* grâce à des tableaux.

F5

(`tty_getw_irq` se trouve dans `giet_2011/app/stdio.c`)

Fonction `tty_getw_irq()`:

- Analysez le code de l'appel système `tty_getw_irq()`.

Lit jusqu'à rencontrer un *"retour chariot"* ou un *"line feed"* ou au bout de 32 caractères lus. Puis convertit la suite de caractères décimaux en string.

- Quels sont les caractères spéciaux qui sont analysés et traités par cet appel système ?

Les caractères spéciaux analysés sont *"retour chariot"* (0x0D) ou *"line feed"* (0x0A). Lorsque l'un des deux est rencontré, la fonction arrête de lire le buffer. Le *"DEL character"* (0x7F) est également analysé et permet d'effacer le dernier caractère lu par un espace.

- Que se passe-t-il si le nombre de caractères décimaux saisis au clavier définit un nombre trop grand pour être codé sur 32 bits ?

Si le nombre de caractères décimaux saisis est trop grand, la fonction efface tous les caractères et retourne 0.

F6

- Les tableaux `_tty_get_buf[]` et `_tty_get_full[]` sont déclarés dans le fichier `drivers.c`. Pourquoi ces variables doivent-elles être déclarées avec l'attribut `volatile` ?

On déclare ces variables en `volatile` pour forcer GCC à faire un accès mémoire lorsque l'on va lire ces tableaux. Sans cela, GCC optimiserait ces variables et les enregistrerait dans un registre qui fausserait la lecture de ceux-ci.

- Dans quel segment mémoire ces variables sont-elles rangées ?

Ces variables sont donc rangées en RAM dans un segment non-cachable qui fait partie du Kernel (`kdata`).

- Pourquoi ce segment doit-il être déclaré non cachable ?

S'il était cachable, cela empêcherait d'effectuer un accès mémoire et on reviendrait au même problème qu'on avait sans l'attribut `volatile`, leurs valeurs ne seraient pas sensibles au fonctionnement asynchrone.

F7

(`proc0`)

```
la $29, _isr_tty_get
sw $29, 12($27) # _vector_interrupt[3] = _isr_tty_get
```

(`proc1`)

```
la $29, _isr_tty_get  
sw $29, 20($27) # _vector_interrupt[5] = _isr_tty_get
```

F8

Pour connaître la position des composants dans le mask, on regarde la description du fichier *tp5_top.cpp* qui dit à quelle entrée IRQ_IN un composant est relié.

On voit que TTY est relié à l'entrée 3 de l'ICU.